# Desenvolvimento Web

Profª Sabrina B. Moreira

# Fundamentos da web 2

2.1 Conceitos de arquitetura de rede

# Modelo centralizado

- A computação centralizada foi predominante nas décadas de 1960 e 1970.
- Quando grandes computadores conhecidos como mainframes **concentravam** todo o **processamento** dos sistemas.

# Modelo centralizado

- Nesse modelo, processamento, armazenamento e controle ficavam **concentrados** em um **único sistema**.

Fonte: technologyuk, 2026

# Modelo centralizado

- Esses computadores eram acessados por meio de “terminais burros”, que possuíam apenas monitor e teclado, sem capacidade própria de processamento.

- Toda a **lógica do sistema** era executada no computador central (mainframe).

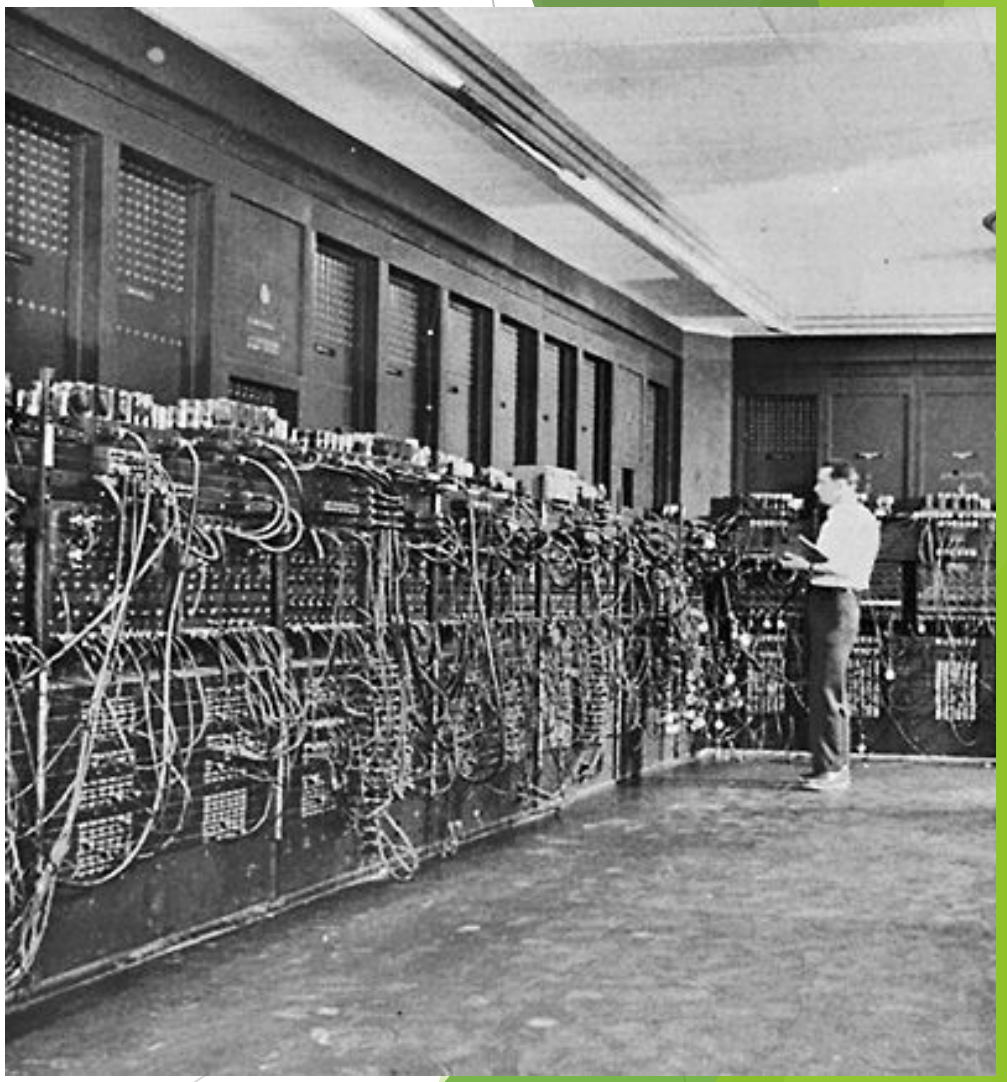
# Características do “terminal burro”

- **Não possui processamento próprio** relevante
- **Não executa lógica** de aplicação
- Apenas envia **entrada** (teclado)
- Apenas exibe **saída** (monitor)

Fonte: technologyuk, 2026

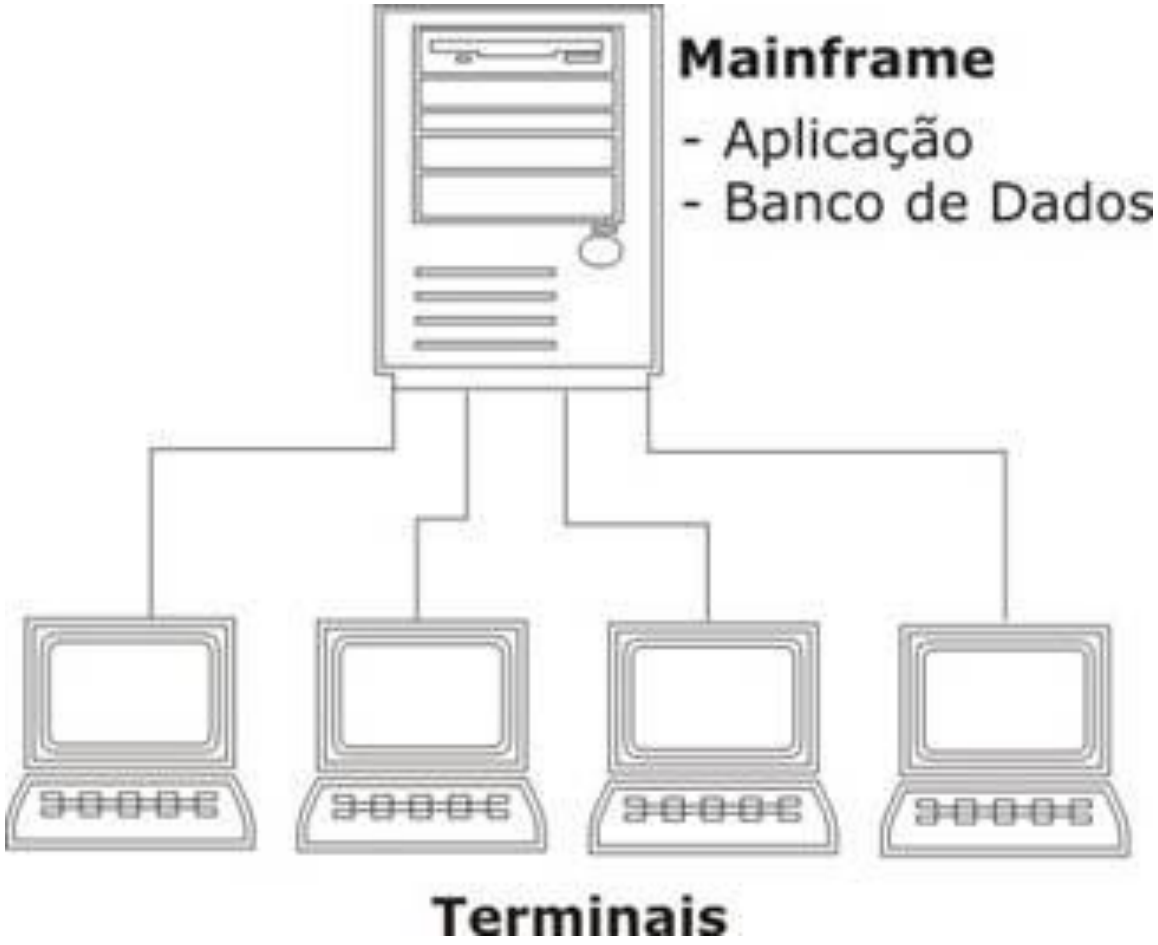
# Características do Mainframe

- Processam grandes volumes de dados e por isso são utilizados por bancos, governos e grandes empresas.
- São construídos para funcionar continuamente com mecanismos de recuperação automática de falhas.
- Pode atender muitos usuários ao mesmo tempo sem afetar o desempenho.
- Os usuários acessam o sistema por meio de **terminais** ou **outros computadores** conectados em rede.



Fonte: infotechba

# Interação Mainframe x terminal




Fonte: battisti

# Modelo centralizado

Fonte: soc, 2013

- Esse paradigma influenciou a evolução da computação moderna e serviu de base para modelos posteriores de compartilhamento de recursos, como os utilizados atualmente na **computação em nuvem.**

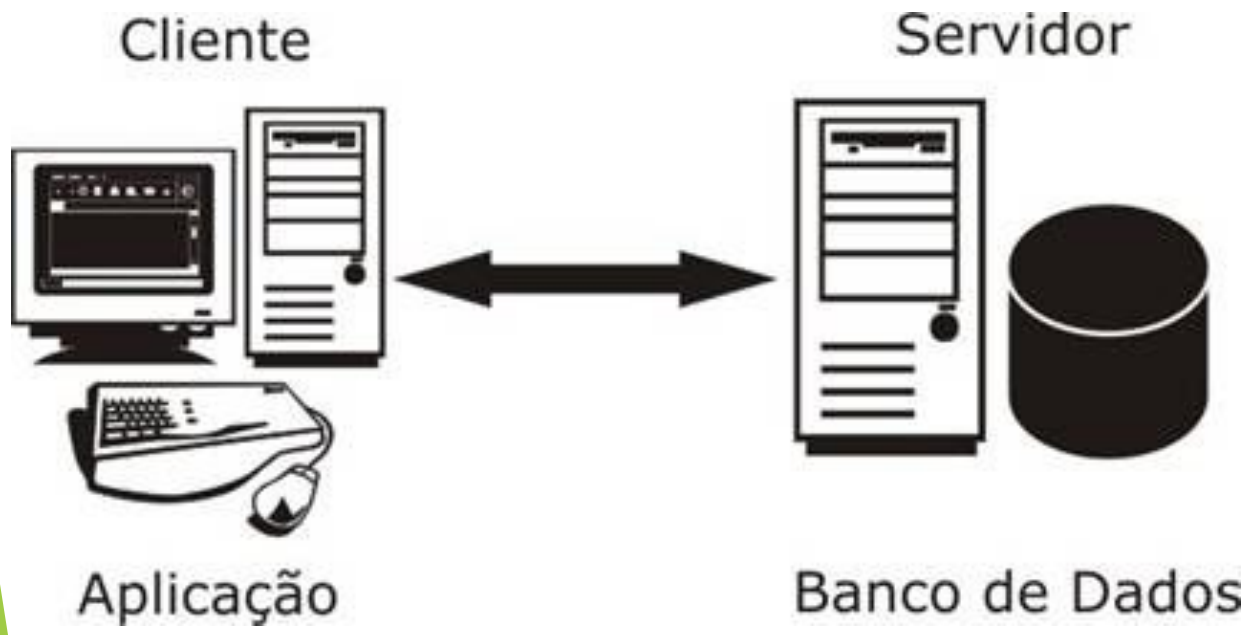
# Recapitulando…

E como já vimos, a criação da ARPANET (1969), surge a necessidade de:

- Compartilhar recursos
- Acessar computadores remotamente
- Distribuir processamento

A popularização das redes e a necessidade de compartilhamento de recursos consolidaram o modelo **cliente-servidor** nas décadas de 1980 e 1990.

# Modelo distribuído



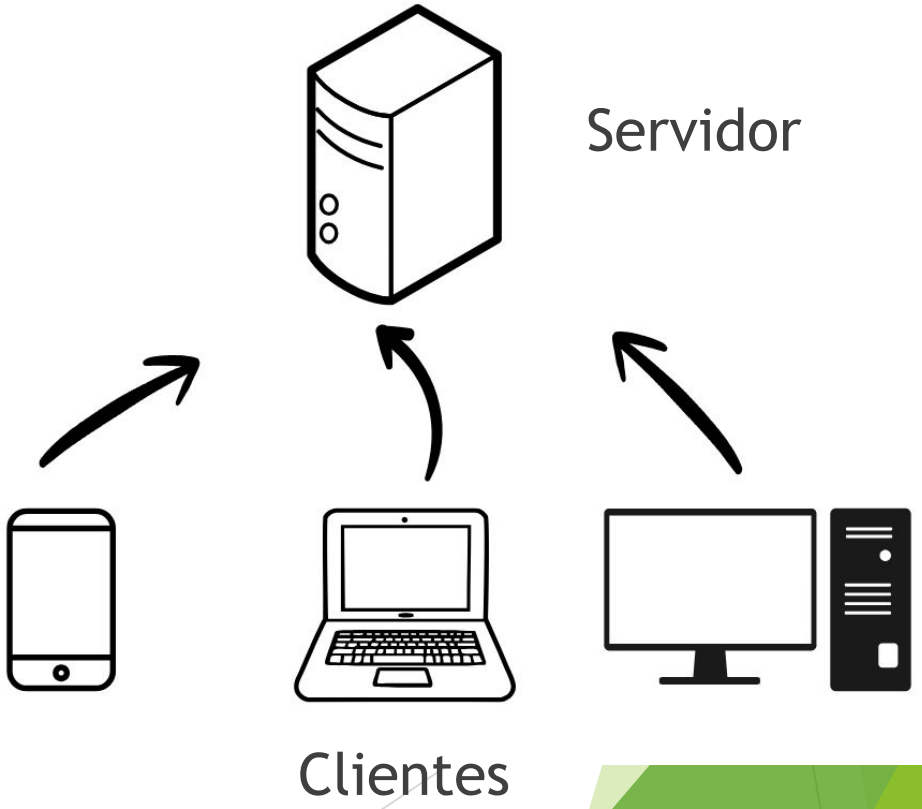

Fonte: battisti

- Caracteriza-se pela <u>execução de processos</u> em **múltiplos computadores interligados por uma rede**, que <u>cooperam</u> entre si para realizar tarefas ou fornecer serviços.

# Modelo distribuído

- Um dos modelos mais comuns de sistemas distribuídos é o modelo **cliente-servidor**.

- No qual clientes solicitam serviços e servidores processam essas solicitações e retornam respostas.




Fonte: autora, 2026

# Modelo distribuído

As principais diferenças entre o modelo **centralizado** e o modelo **distribuído** são:

- No modelo centralizado, o terminal não executa lógica de aplicação localmente e **apenas exibe** o processamento realizado pelo servidor, não tendo autonomia.

- Já no modelo distribuído, o cliente **compartilha responsabilidades** com o servidor, podendo realizar alguns processamentos localmente, como validação de formulário ou manipulação de interface, tendo autonomia parcial.

# Cliente-Servidor: o que é cliente

Cliente:

- Inicia comunicação
- Solicita recursos ao servidor
- Faz execuções na máquina do usuário
- Processa interface gráfica
- Valida dados
- Executa parte da lógica da aplicação
- Pode armazenar dados temporariamente

# Cliente-Servidor: exemplos de clientes

- Um **navegador** fazendo requisição HTTP
- Um **app mobile** chamando uma API
- Um **software desktop** solicitando informações a um servidor.
- Um **backend** consumindo outra API
- Um **script automatizado** acessando um serviço
- Um **microserviço** chamando outro microserviço

# Cliente-Servidor: exemplos de clientes

- Navegadores
- Spotify
- WhatsApp
- Microsoft Outlook

Fonte: autora, 2026

# Cliente-Servidor: o que é cliente

Por que são clientes?

- Enviam requisições
- Recebem dados

Ou seja, seu funcionamento **depende** de um ou mais servidores.

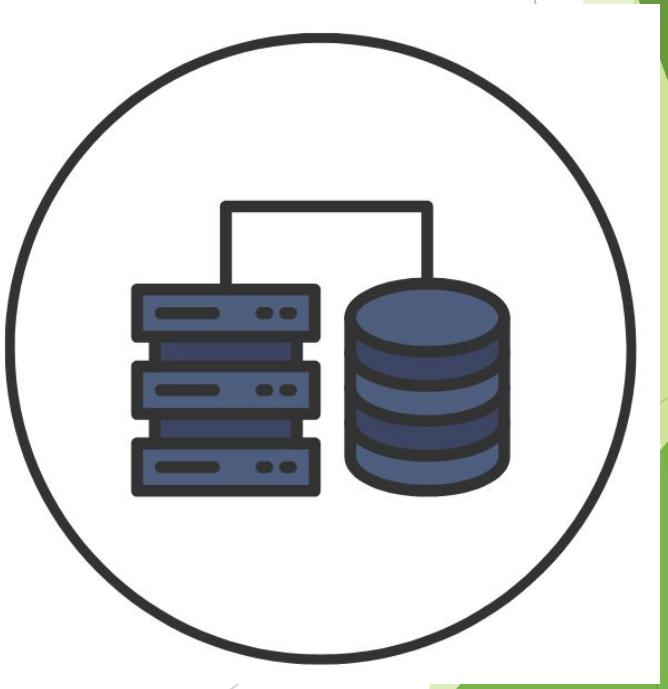
Mas ser cliente <u>não implica</u> ter necessariamente uma **interface gráfica**.

# Cliente-Servidor: o que é servidor

- É um processo especializado que **disponibiliza serviços na rede**, permanecendo ativo e acessível para atender múltiplas requisições concorrentes de clientes, seguindo protocolos específicos de comunicação.

Principais características

- Fica aguardando requisições
- Fornece serviços ou recursos
- Pode atender múltiplos clientes simultaneamente



Fonte: autora, 2026

# Cliente-Servidor: exemplos de servidores

Exemplos:

- **Servidor Web** (Apache, Nginx) (recebe requisições, armazena arquivos, encaminha requisições para aplicações)

- **Banco de dados** (Sistema de armazenamento que atende outros servidores)

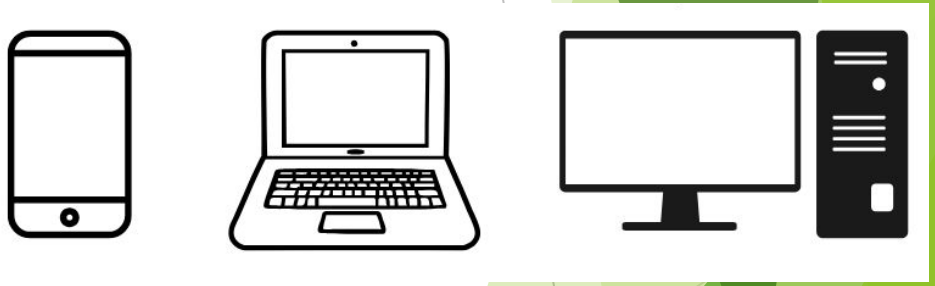
# Cliente-Servidor: cliente

- O cliente é a **aplicação** que roda no dispositivo do usuário (navegador, app mobile, app desktop). O dispositivo (celular, notebook e computador) é apenas o **meio físico**.

- As **aplicações web** são clientes do servidor de **aplicação**, que guarda a regra de negócio e faz os processamentos necessários.

- E o servidor da **aplicação**, é cliente do **banco de dados**, que armazena as informações persistentes.

Meios físicos de acesso



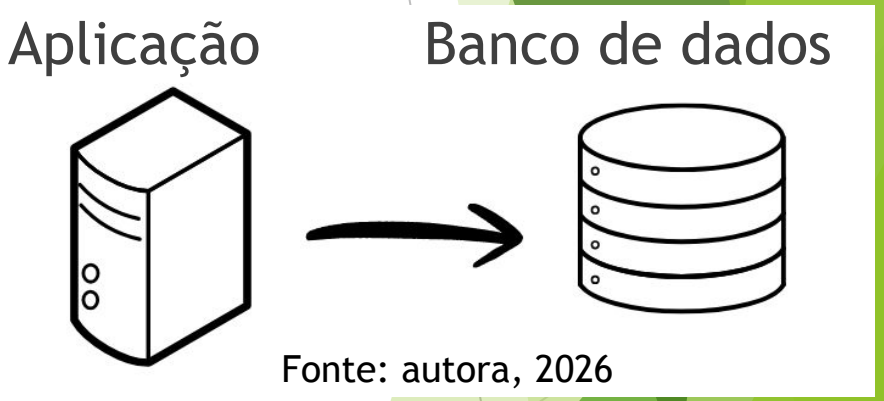
Fonte: autora, 2026

# Cliente-Servidor: servidor

- O **servidor** segue funcionando de forma autônoma, aguardando aparecer clientes solicitando algo a eles.

Ele:

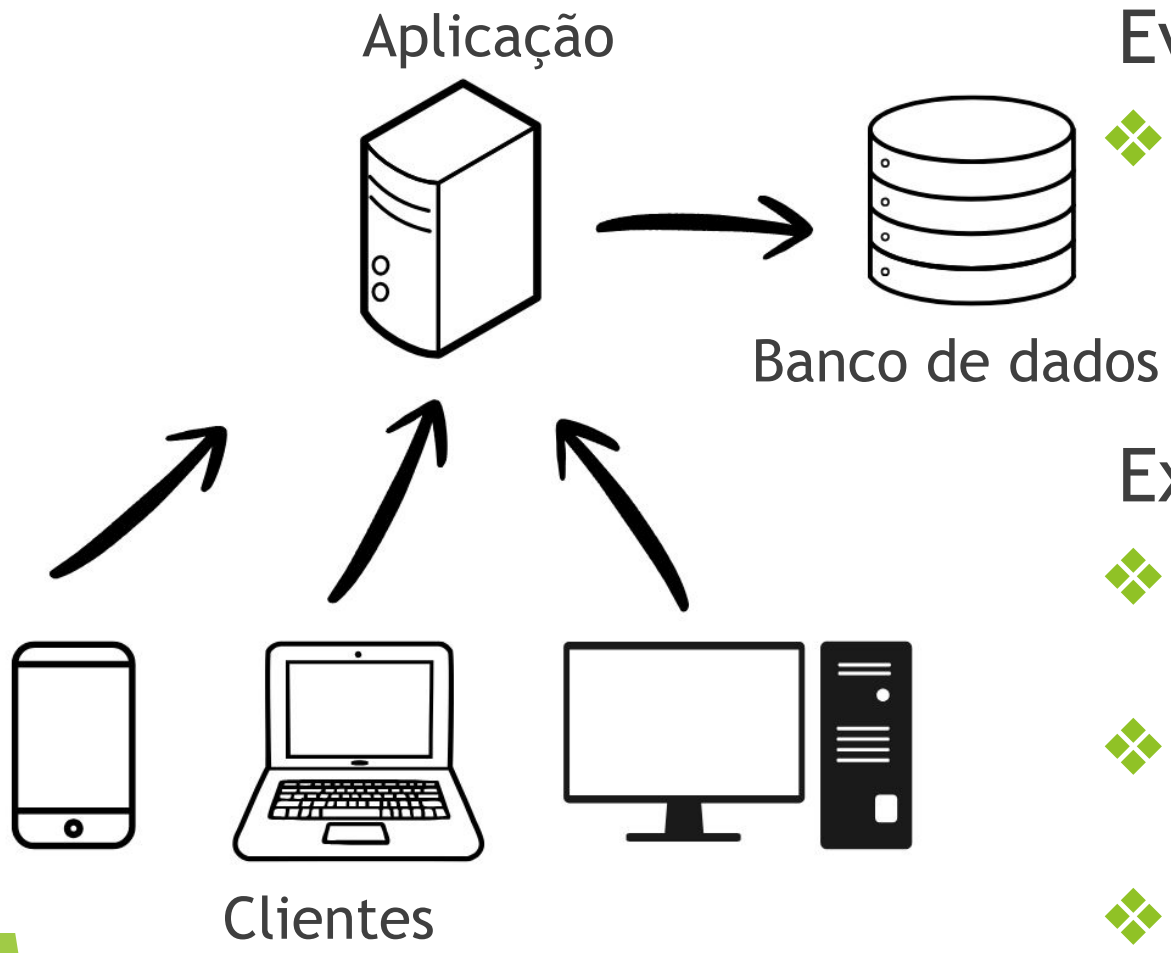
- Fica em execução contínua
- Escuta requisições em uma porta específica
- Responde sob demanda



Fonte: autora, 2026

O <u>servidor não depende</u> de um **cliente** específico para existir, mas depende de clientes para cumprir sua função.

# Cliente-Servidor: arquitetura em camadas




Fonte: autora, 2026

Evolução da Arquitetura:

- Hoje temos estruturas baseadas em diversas camadas.

Exemplo clássico 3-tier:

- Camada de **Apresentação**
- Camada de Lógica de **Negócio**
- Camada de **Dados**

# Cliente-Servidor: exemplo Spotify

Quem é o cliente?

- O app do Spotify no celular
- O navegador no computador
- O app da Smart TV

O dispositivo do usuário é o cliente.

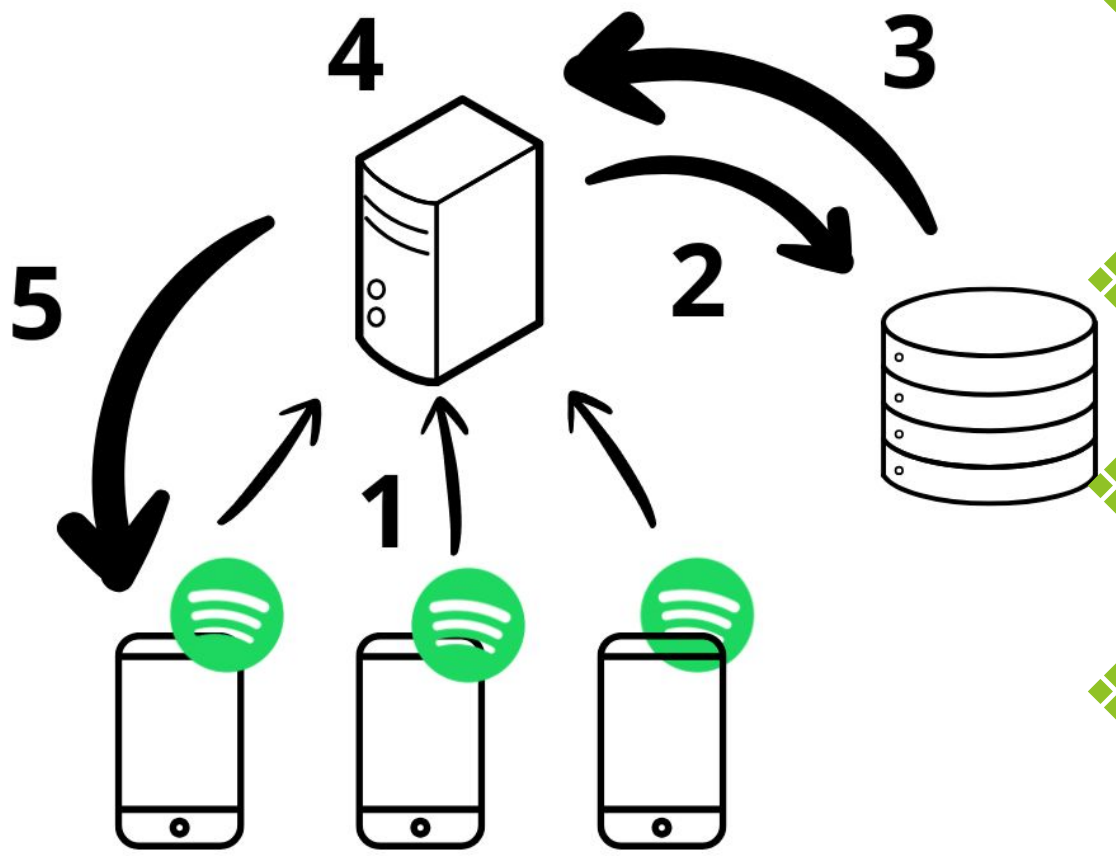
Quem é o servidor?

- Os servidores do Spotify espalhados pelo mundo
- Data centers que armazenam músicas e dados de usuários

Esses servidores enviam:

- A música (streaming)
- Dados do perfil
- Playlists
- Recomendações

# Exemplo Spotify: O que acontece tecnicamente?




Fonte: autora, 2026

- Cliente envia requisição: “Quero tocar essa música”
- Servidor autentica usuário
- Servidor envia ao áudio em pequenos blocos
- Cliente reproduz enquanto continua recebendo dados

# Definição de rede x definição arquitetural

**Cliente (Arquitetura de redes)**

Cliente é o processo que inicia uma requisição a um servidor.

- Pode ser navegador
- Pode ser app mobile
- Pode ser software desktop
- Pode ser outro servidor

Cliente é um papel na comunicação, não um tipo de interface.

**Frontend (Arquitetura de software)**

Frontend é:

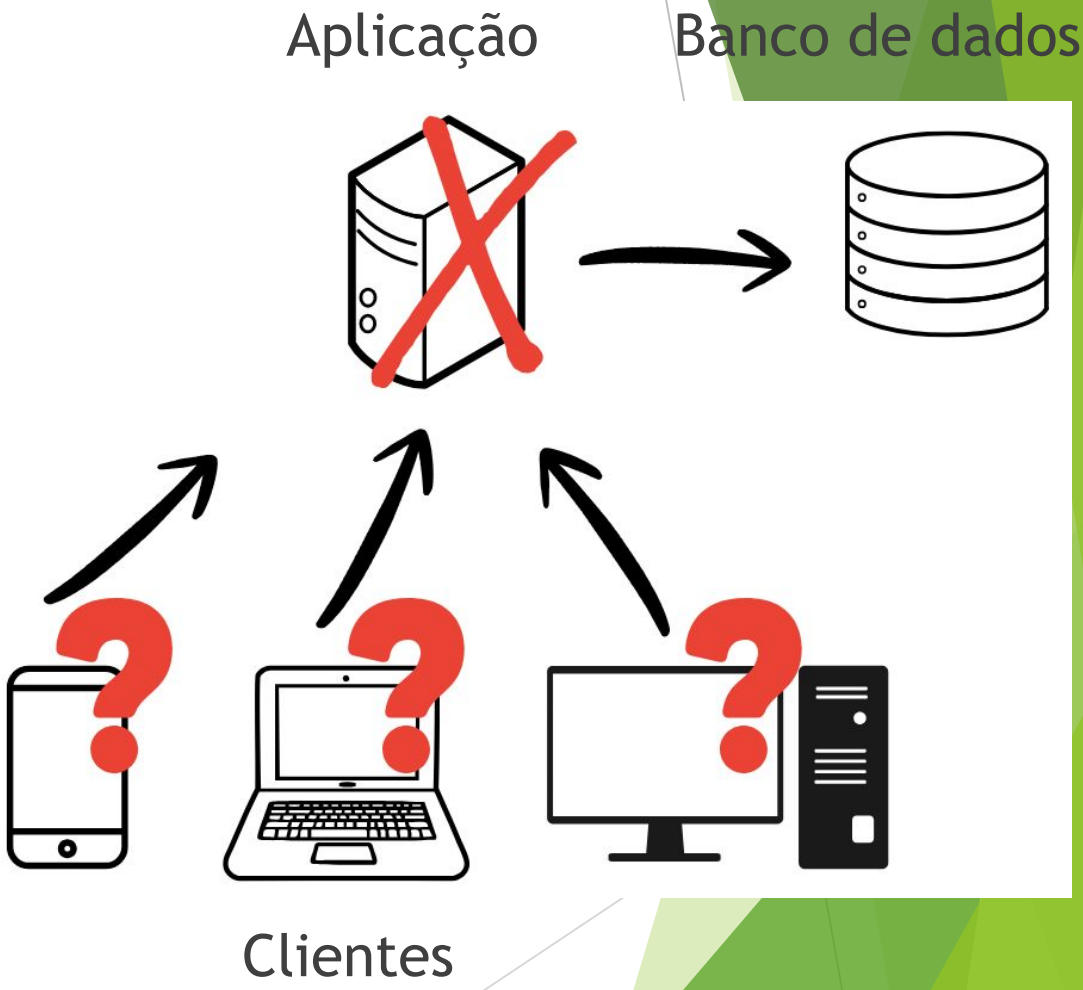
- A camada de interface com o usuário
- Responsável pela apresentação
- Parte da aplicação que roda no dispositivo do usuário

O frontend de uma aplicação web geralmente é cliente.

# Cliente-Servidor: arquitetura em camadas

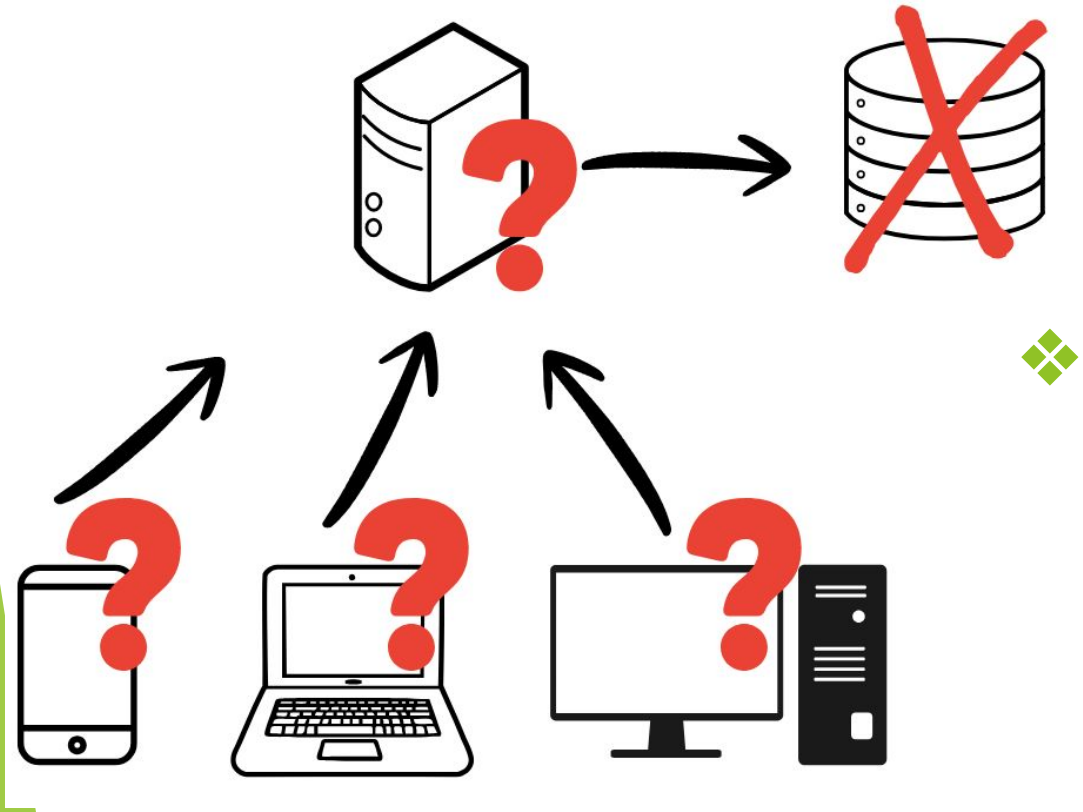
- Os **clientes** dependem das informações processadas enviadas pelos servidores para executar suas rotinas.

- Se o servidor falhar, o cliente será impactado.




Fonte: autora, 2026

# Cliente-Servidor: arquitetura em camadas



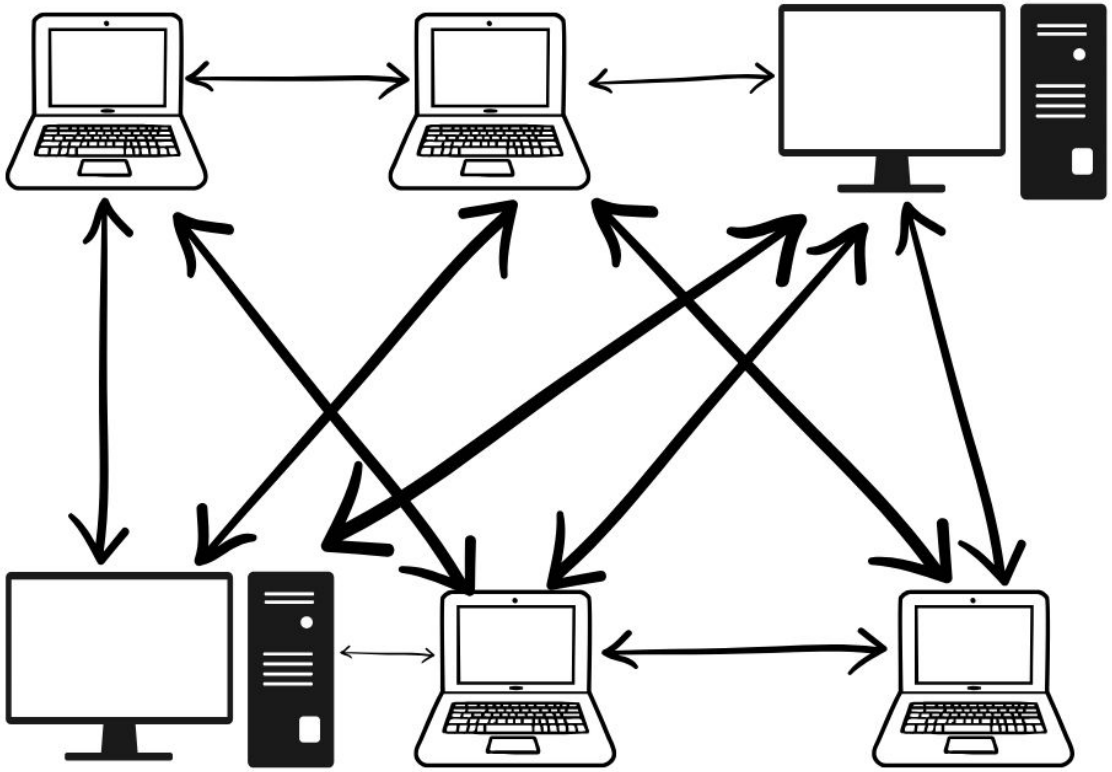
Fonte: autora, 2026

- No modelo cliente-servidor existe dependência funcional.
- Se o servidor parar ou ficar indisponível ou responder com erro, <u>o cliente não consegue concluir sua operação</u>.

# Fundamentos da web 2

2.2 Outras arquiteturas

# Arquitetura Peer-to-Peer (P2P)



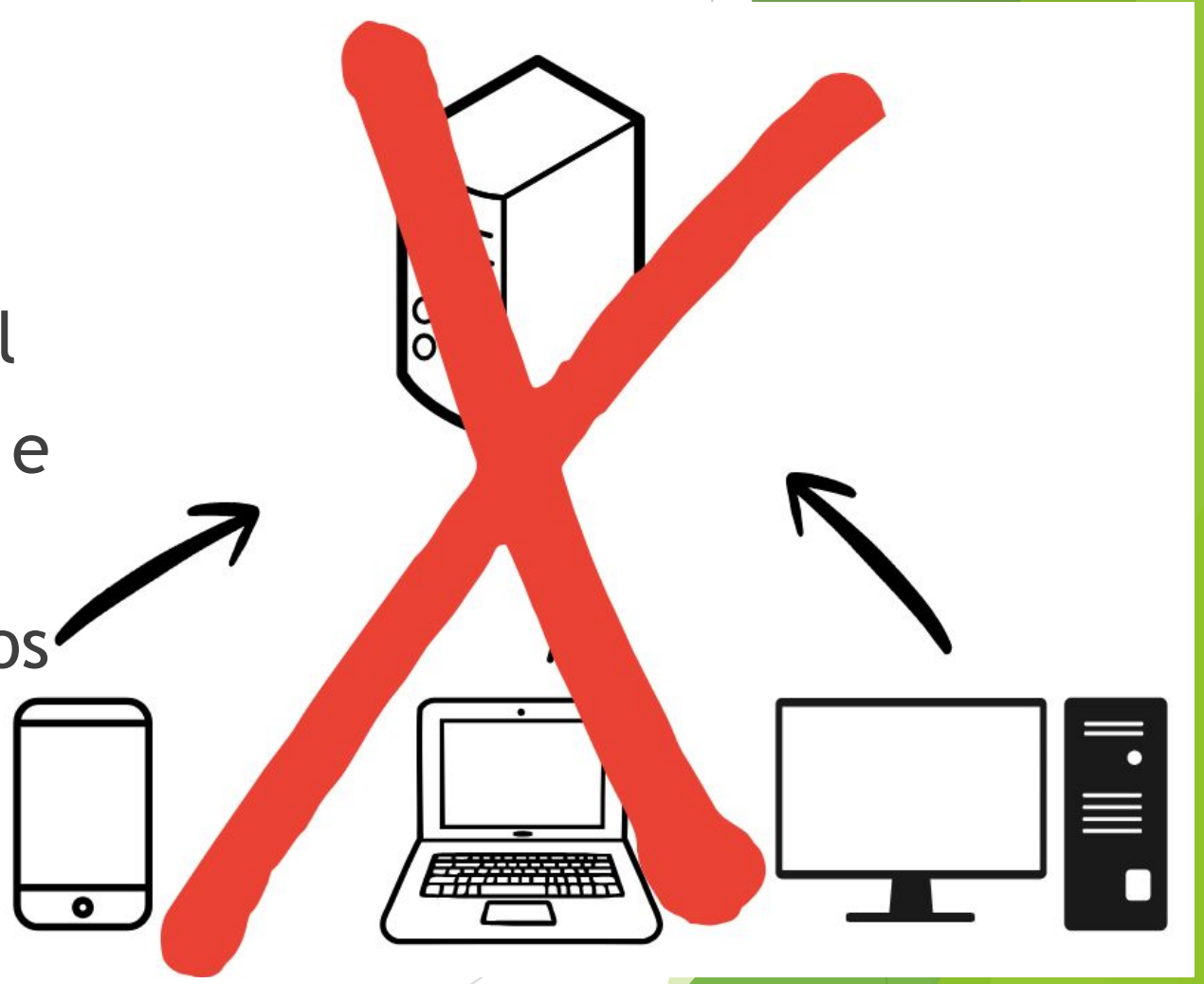
Fonte: autora, 2026

No modelo peer-to-peer, todos os **nós** são chamados de **peers** (pares) que podem atuar tanto como cliente quanto como servidor ao mesmo tempo.

# Arquitetura Peer-to-Peer (P2P)

Ou seja:

- Não existe um servidor central
- Cada máquina pode receber e fornecer recursos para outras
- O tráfego é distribuído entre os pares



Fonte: autora, 2026

# Arquitetura Peer-to-Peer (P2P)

Características:

- Descentralização (sem ponto único de falha)
- Escalabilidade natural, quanto mais peers, mais recursos
- Utilização eficiente de banda e armazenamento

Desafios:

- Controle e segurança mais complexos
- Coordenação dos nós exige protocolos eficientes

# Arquitetura Peer-to-Peer (P2P)

Fonte: autora, 2026

- BitTorrent é um **protocolo de comunicação P2P** criado por Bram Cohen para distribuir dados de forma eficiente.

- Softwares clientes que seguem esse protocolo permitem que cada participante atue simultaneamente como cliente e servidor, compartilhando partes do arquivo com outros peers, sem depender de um servidor central para a distribuição dos dados.

# Arquitetura Peer-to-Peer (P2P)

Protocolo = conjunto de regras que define:

- como os peers se encontram
- como os arquivos são divididos
- como os blocos são trocados
- como a integridade é verificada

Ele **implementa** arquitetura P2P, mas ele não é a arquitetura em si.

- Arquitetura P2P é um modelo conceitual.
- BitTorrent é uma implementação concreta desse modelo.

# Arquitetura Peer-to-Peer (P2P)

Fonte: autora, 2026

# Arquitetura Peer-to-Peer (P2P)

**1 Fragmentação do arquivo**

- O arquivo a ser compartilhado é dividido em vários pedaços (chunks ou pieces).

Isso é essencial porque:

- Permite download simultâneo de múltiplos peers.
- Permite que um peer compartilhe mesmo antes de terminar o download.

**2 Entrada na rede (Swarm)**

- O conjunto de peers que compartilham um mesmo arquivo é chamado de swarm.

Dentro do swarm existem:

- Seeder: Peer que possui o arquivo completo.
- Leecher: Peer que ainda está baixando.

# Arquitetura Peer-to-Peer (P2P)

**3 Comunicação entre peers**

Ao entrar na rede:

- O peer descobre outros peers (via tracker ou DHT).
- Solicita pedaços diferentes para diferentes peers.
- Envia pedaços para quem solicitar.

Isso gera um efeito muito importante:

- Quanto mais pessoas baixando, mais rápida tende a ser a distribuição.
- Diferente do modelo cliente-servidor, onde muitos usuários sobrecarregam um único servidor.

# Arquitetura Peer-to-Peer (P2P)

Clientes BitTorrent (softwares):

- qBittorrent
- uTorrent

Esses são programas que utilizam o protocolo.

Fonte: autora, 2026

# Cliente-Servidor x Peer-to-Peer

No modelo cliente-servidor:

- Papéis bem definidos
- Um servidor central armazena tudo
- Todos os clientes dependem dele

No P2P:

- Todos podem ser cliente e servidor
- Não existe servidor central para os dados
- Os próprios usuários distribuem os arquivos

# Fundamentos da web 2

2.3 Arquiteturas de rede modernas

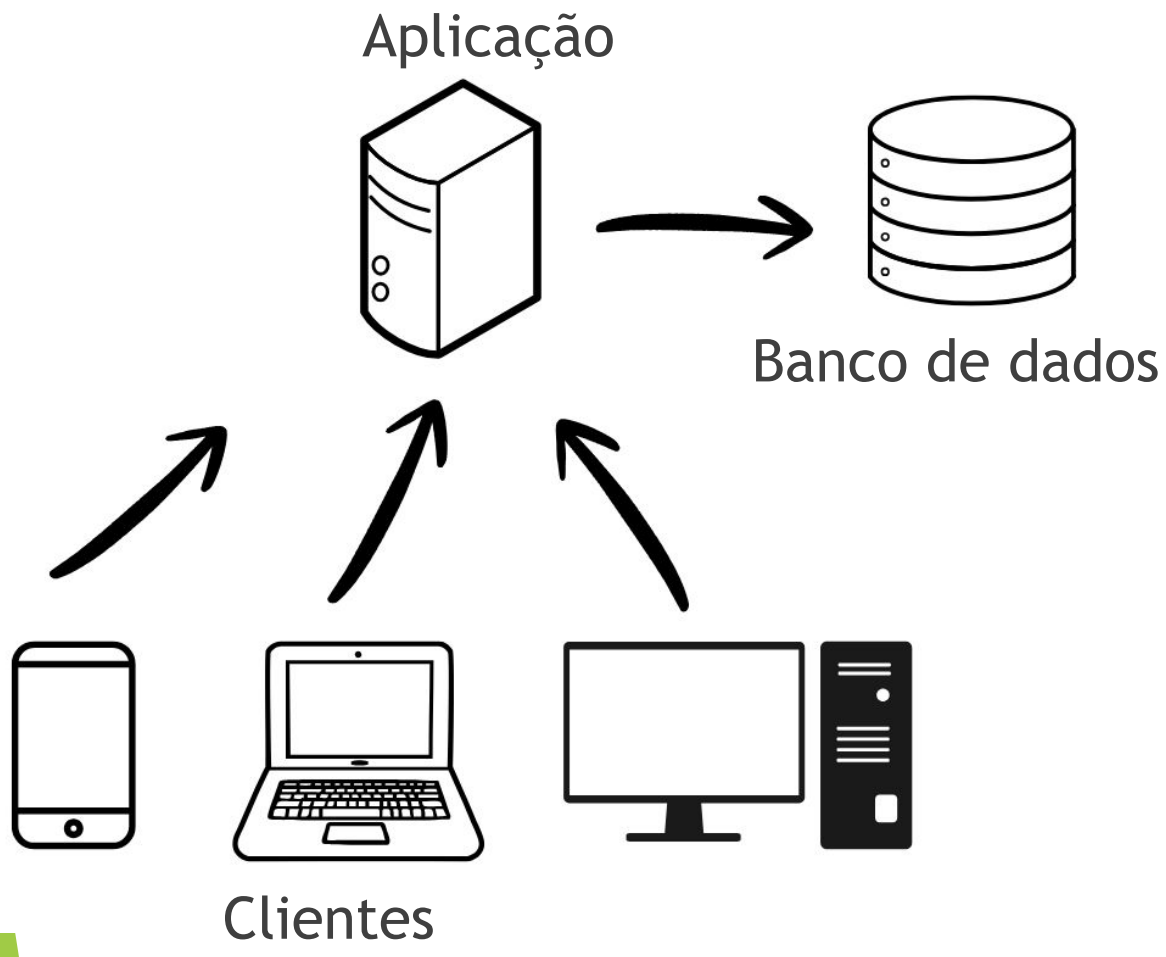
# Recapitulando…

**Cliente-Servidor Tradicional (Monolítico)**

- Um servidor central
- Toda lógica concentrada
- Um banco de dados central

Exemplo: sistemas corporativos antigos.

# Cliente-Servidor Tradicional




Fonte: autora, 2026

- **Pontos positivos**: simples de desenvolver, fácil de implantar e entender.

- **Pontos negativos**: difícil de escalar, maior risco de indisponibilidade e alto acoplamento entre os módulos.

# Arquitetura modernas

**Cliente-Servidor em Nuvem**

Exemplo:

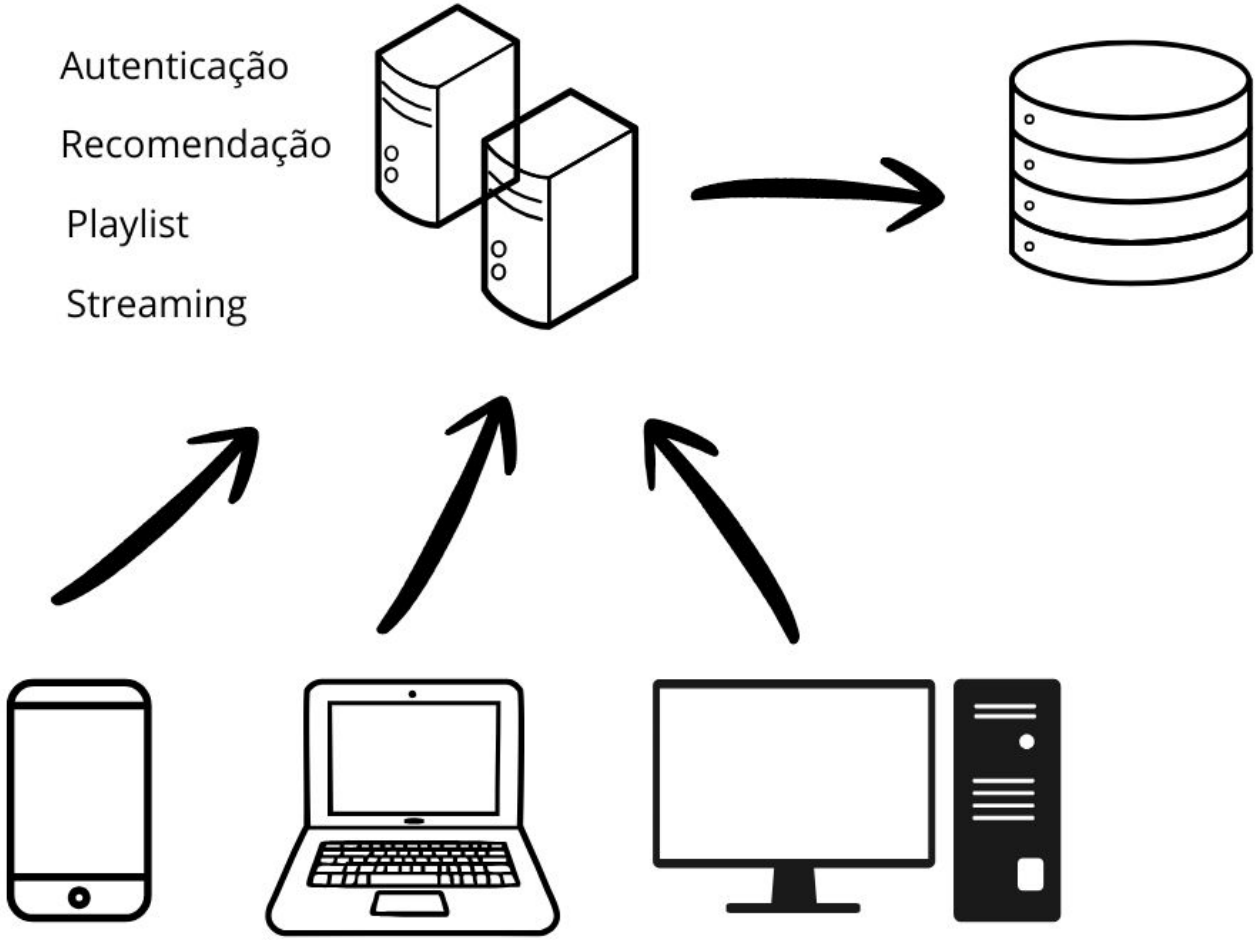
- Netflix
- Google Drive

Na nuvem, o servidor não é uma única máquina, mas milhares de servidores distribuídos que trabalham juntos para processar requisições de forma rápida, escalável e confiável.

# Cliente-Servidor em Nuvem

- **Pontos positivos**: alta escalabilidade e disponibilidade, fácil expansão pelo modelo infraestrutura como serviço e paga-se pelo uso.

- **Pontos negativos**: dependência alta do provedor principalmente com abstração da infra, custos podem sair do controle facilmente e maior nível de conhecimento do provedor em questão.




Fonte: autora, 2026

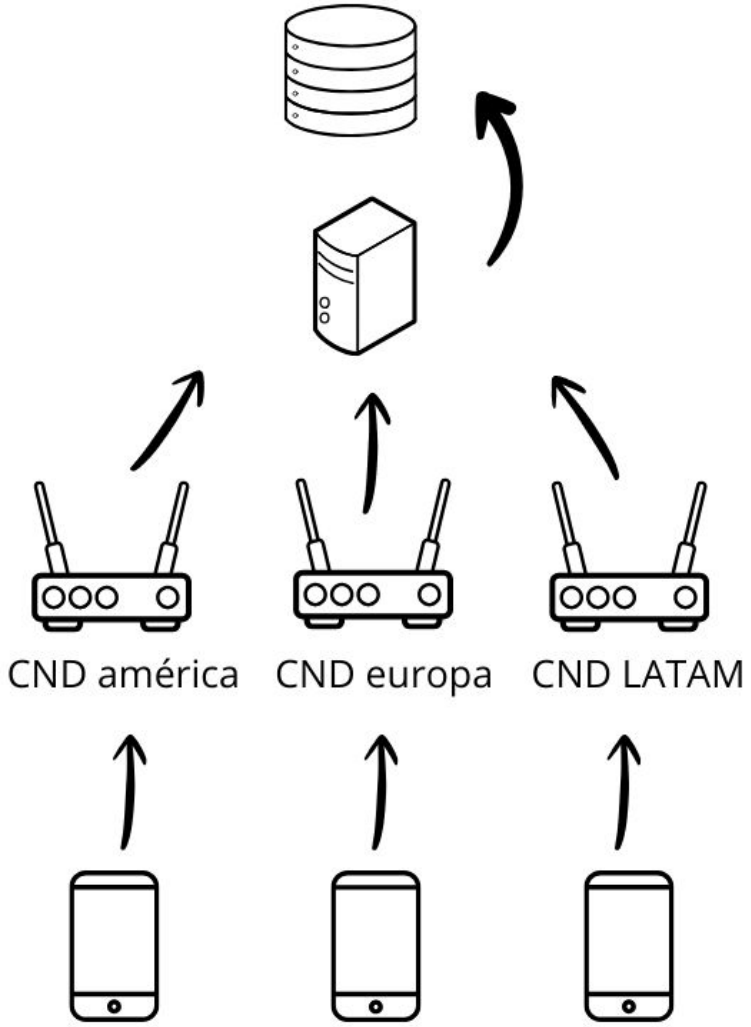
# Cliente-Servidor: arquiteturas modernas

**Cliente-Servidor com CDN**

- São servidores espalhados geograficamente para reduzir latência. O Spotify e Netflix usam CDN.

- Apenas o <u>conteúdo estático</u> (vídeo, imagens, áudio, arquivos) é replicado em servidores próximos do usuário. O objetivo é reduzir latência, melhorar desempenho, aliviar carga do servidor principal

# Cliente-Servidor com CDN




Fonte: autora, 2026

- **Pontos positivos**: melhor performance global e melhor experiência dos usuários globais.
- **Pontos negativos**: exige investimento e pessoal capacitado para sua implementação.

# Cliente-Servidor: arquiteturas modernas

**Cliente-Servidor com Micro Serviços**

- Empresas dividem um sistema grande em vários serviços menores e conectados.

Exemplo hipotético do Spotify:

- Serviço de autenticação
- Serviço de playlists
- Serviço de recomendação
- Serviço de streaming

Cada um é um <u>servidor especializado</u> em um tipo de **processamento**.

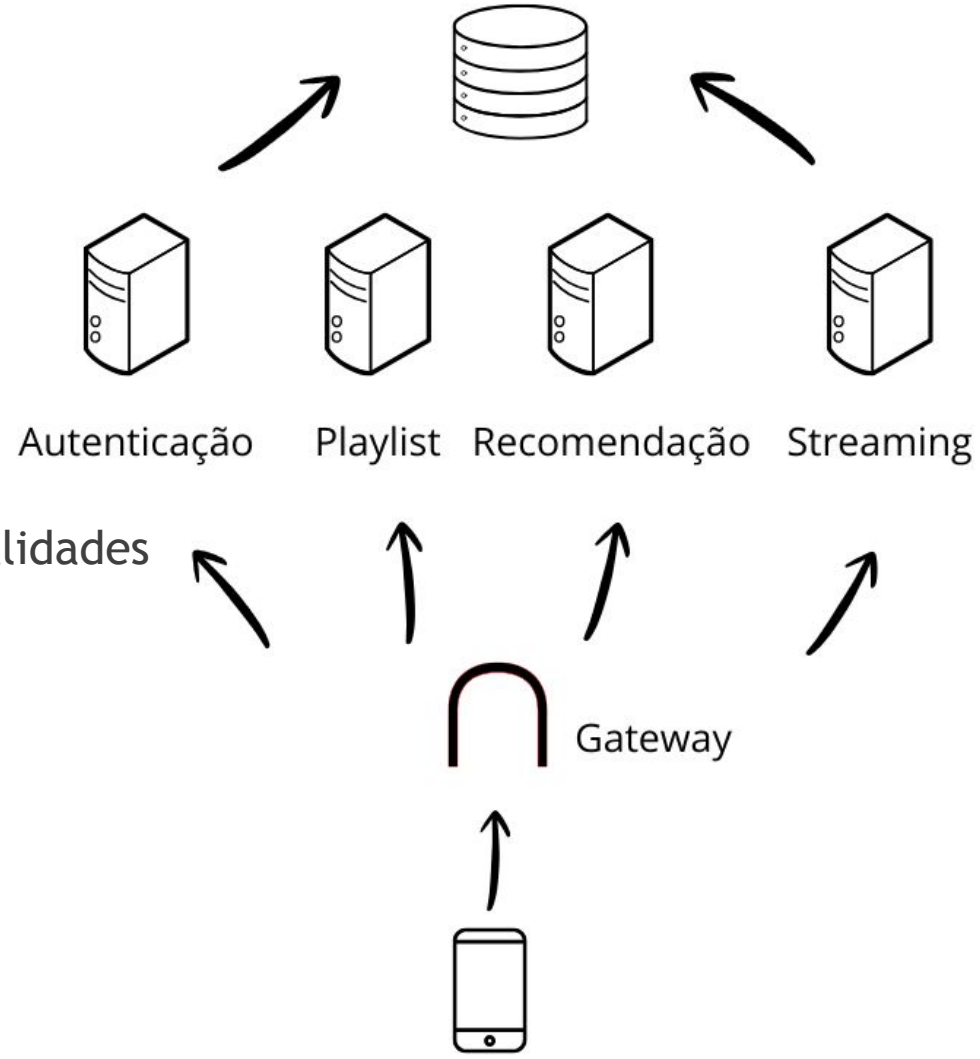
# Cliente-Servidor: arquiteturas modernas

**Pontos positivos:**

- Desacoplamento de serviços
- Menos sucessão a falha generalizada
- Maior reuso dos serviços para diferentes aplicações
- Maior flexibilidade tecnológica
- Maior velocidade de desenvolvimento de novas funcionalidades

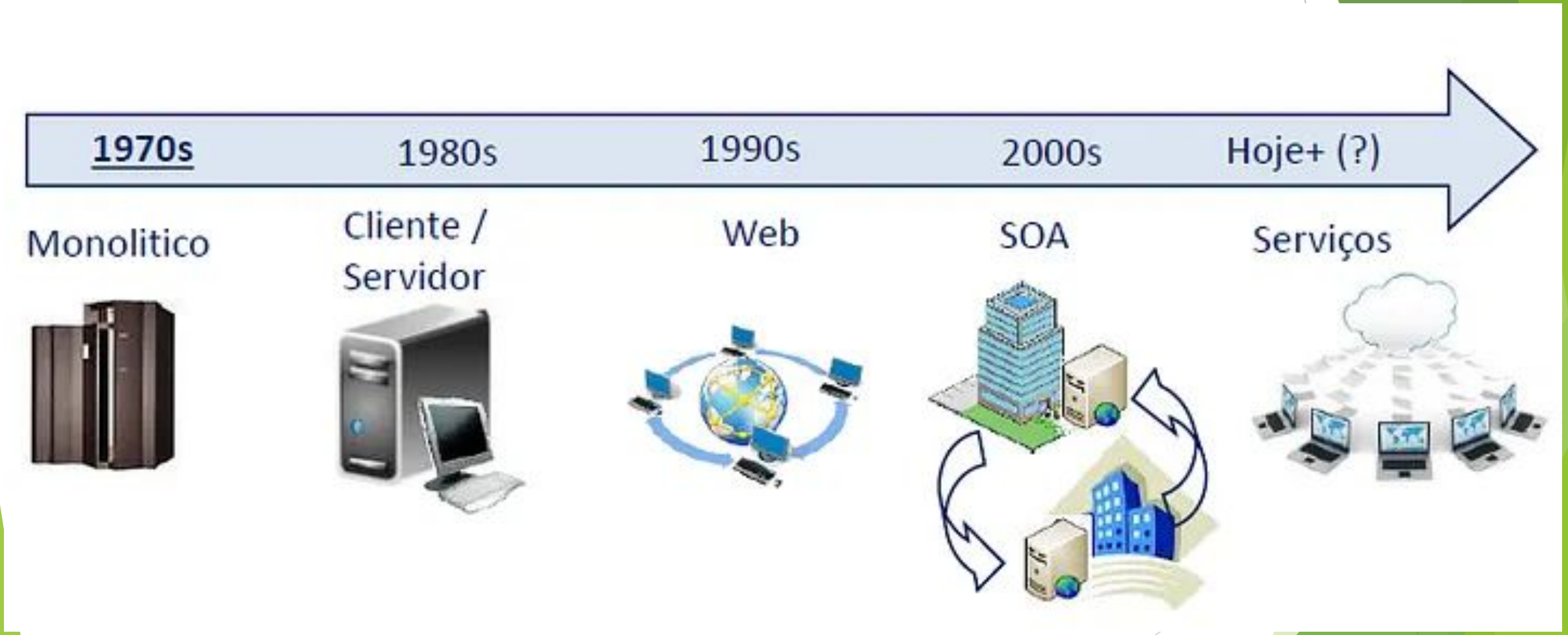
**Pontos negativos:**

- Altíssima complexidade de gerenciamento
- Difícil monitoramento e atualização
- Complexa arquitetura
- Maior custo operacional de infra e time




Fonte: autora, 2026

# Arquiteturas de sistemas computacionais




Fonte: Oliveira, 2017

# Arquiteturas de sistemas computacionais

- No modelo monolítico, todo o sistema roda em **um único computador central**, normalmente um mainframe.
- Na arquitetura cliente-servidor o sistema passa a ser **dividido** em duas partes, a parte que roda no computador do usuário (**cliente**) e o **servidor** que fornece serviços e dados.
- Arquitetura web: O navegador passa a ser cliente e o sistema passa a ser acessado por **páginas web** gerado por um servidor.
- SOA (Service-Oriented Architecture): o sistema passa a ser divido em **serviços menores e reutilizáveis** por diversas aplicações.
- Nuvem e micro serviços: separa o sistema em **serviços menores**, **especializados** e **independentes**, geralmente em servidores diferentes na nuvem.

# Dúvidas?

# Atividade 2 - Questionário

- Responder questionário referente aos assuntos da aula de hoje.
- **Data de entrega**: até o final da aula de HOJE
- **Forma de entrega**: manuscrito físico.
- **Execução:** duplas ou trios (colocar nome de todos)
- **Fonte de consulta**: slides, grupo e internet.

# Referência da aula


- TECHNOLOGYUK. Mainframe computers. TechnologyUK.net, s.d. Disponível em: https://www.technologyuk.net/computing/computer-hardware/mainframe-computers.shtml. Acesso em: 26 fev. 2026.

- TANENBAUM, Andrew S.; WETHERALL, David J. Computer Networks. 5. ed. Boston: Pearson, 2011.

- KUROSE, James F.; ROSS, Keith W. Computer Networking: A Top-Down Approach. 7. ed. Boston: Pearson, 2017.

- ANDERSON, Ross. Security Engineering: A Guide to Building Dependable Distributed Systems. 2. ed. Indianapolis: Wiley, 2008.

- MOCKAPETRIS, Paul. Domain names – implementation and specification. RFC 1035. 1987.

# Referência da aula


- ICANN. Internet Corporation for Assigned Names and Numbers. Disponível em: https://www.icann.org

- NIC.BR. Núcleo de Informação e Coordenação do Ponto BR. Disponível em: https://www.nic.br

- MILETTO, Evandro M.; BERTAGNOLLI, Silvia C. Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, javascript e PHP. (Tekne). Porto Alegre: Bookman, 2014. E-book. p.72. ISBN 9788582601969. Disponível em: https://integrada.minhabiblioteca.com.br/reader/books/9788582601969/. Acesso em: 08 mar. 2026.

univille

univille.br